



INFORME TÉCNICO

CODIFICACIÓN Y VALIDACIÓN DE API RESTFUL PARA EL PROYECTO "EDUTRACK"

EVIDENCIA: GA6-220501123-AA2-EV01

PROGRAMA DE FORMACIÓN: Técnico en Programación de Aplicaciones y Servicios para la Nube / Tecnólogo en Análisis y Desarrollo de Software

FICHA DE CARACTERIZACIÓN: 3335994 / 228122

PRESENTADO POR (EQUIPO DE DESARROLLO):

- Santiago Barbosa
- Emanuel Romero
- Rebeca Rivera
- Paula Martínez

PRESENTADO A:

Jorge Callegas (Instructor SENA)

MAYO, 2026



1. TABLA DE CONTENIDO

1. Introducción

2. Objetivos del Informe

Objetivo General

Objetivos Específicos

3. Entorno de Desarrollo y Configuración Tecnológica

4. Estructura Lógica y Código Fuente Implementado

A. Gestión de Fichas Académicas (fichas.service.ts)

B. Servicio de Autenticación y Cifrado (auth.service.ts)

5. Mapeo de Endpoints y Rutas del Servidor (Logs del Sistema)

6. Evidencias de Pruebas Técnicas en Postman

Caso de Prueba 01: Consulta de Fichas Educativas (GET /api/fichas)

Caso de Prueba 02: Autenticación Exitosa de Administrador (POST /api/auth/login)

Caso de Prueba 03: Acceso Protegido con Bearer Token (GET /api/fichas + JWT)

7. Conclusiones

8. Referencias y Bibliografía



2. Introducción

El presente informe técnico documenta el proceso de codificación, levantamiento de servicios y validación de la API RESTful diseñada para el sistema de gestión académica **EduTrack**. Enmarcado en las buenas prácticas de desarrollo ágil y arquitecturas escalables para la nube, se ha implementado el backend utilizando el entorno de ejecución **Node.js** y el framework **NestJS**. La validación de los endpoints, la persistencia simulada de los datos y el flujo de seguridad se exponen detalladamente a través de capturas de pruebas controladas en la plataforma **Postman**.

3. Objetivos del Informe

Objetivo General

Desarrollar y codificar una API RESTful funcional empleando un lenguaje y framework orientados a objetos que permita el intercambio seguro de datos en formato JSON, de acuerdo a los requerimientos del software establecidos en la guía de actividad **GA6-220501123-AA2-EV01**.

Objetivos Específicos

- Estructurar el backend del aplicativo mediante la arquitectura modular, controladores y servicios de NestJS.
- Implementar un sistema de autenticación de usuarios basado en tokens firmados (JWT).
- Validar la conectividad de las rutas mediante peticiones HTTP en Postman, garantizando la correcta recepción de códigos de estado (*Status Codes*).



4. Entorno de Desarrollo y Configuración Tecnológica

La implementación del software se realizó bajo el siguiente entorno tecnológico verificado en las estaciones de trabajo del equipo de desarrollo:

- **Sistema Operativo:** Linux Ubuntu, corriendo sobre una estación de trabajo HP EliteBook 840 G6.
- **Herramienta de Edición (IDE):** Visual Studio Code (VS Code).
- **Estructura Base del Framework:** NestJS empleando TypeScript como lenguaje de tipado estático, facilitando el uso de decoradores (`@Injectable()`, `@Controller()`, etc.).
- **Cliente de Pruebas de API:** Postman Desktop — Workspace del proyecto EduTrack.



5. Estructura Lógica y Código Fuente Implementado

A. Gestión de Fichas Académicas (fichas.service.ts)

Se programaron los métodos lógicos para responder a las solicitudes de la entidad "Fichas", mapeando de manera controlada un arreglo de objetos JSON con datos reales del centro formativo. La estructura de la respuesta inyectada contiene:

- **ID 1:** Código 220501 — Tecnólogo en Análisis y Desarrollo de Software.
- **ID 2:** Código 220502 — Técnico en Control de Procesos Electrónicos.

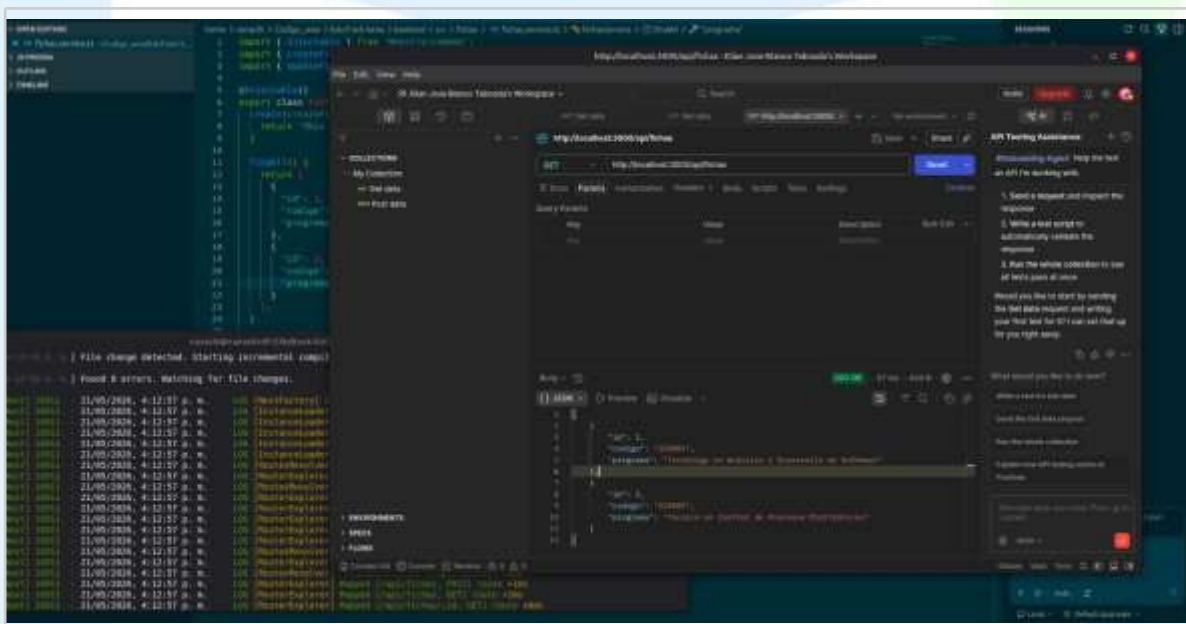


Figura 1. VS Code (fichas.service.ts) y Postman mostrando el GET /api/fichas — respuesta 200 OK con datos JSON de las fichas académicas.

B. Servicio de Autenticación y Cifrado (auth.service.ts)

Como parte de los requerimientos de seguridad para los servicios en la nube, se estructuró la clase AuthService utilizando la inyección de los servicios UsuariosService y JwtService. El método de login realiza las siguientes acciones:

- Busca el usuario por su correo institucional mediante `findByEmail()`.
- Valida existencia del usuario y coincidencia de contraseña; lanza `UnauthorizedException` si falla.
- Extrae la contraseña del objeto resultado por seguridad (destructuring).
- Construye el `payload` con `{ sub: result.id, email: result.email }`.
- Retorna el usuario, el mensaje de éxito y el `access_token` firmado por `JwtService.sign()`.



```
auth.service.ts
import { Injectable, UnauthorizedException } from '@nestjs/common';
import { UsuariosService } from '../usuarios/usuarios.service';
import { LoginDto } from '../dto/login.dto';
import { JwtService } from '@nestjs/jwt'; // 1. <-- NUEVO IMPORT

@Injectable()
export class AuthService {
  // 2. <-- Inyectamos el JwtService en el constructor
  constructor(
    private readonly usuariosService: UsuariosService,
    private readonly jwtService: JwtService
  ) {}

  // Método de login
  login(loginDto: LoginDto) {
    // 1. Buscamos el usuario por su correo
    const usuario = this.usuariosService.findByEmail(loginDto.email);

    // 2. Validamos que el usuario exista y que la contraseña coincida
    if (!usuario || usuario.password !== loginDto.password) {
      throw new UnauthorizedException('Credenciales inválidas');
    }

    // 3. Extraemos la contraseña para no devolverla por seguridad
    // eslint-disable-next-line @typescript-eslint/no-unused-vars
    const { password, ...result } = usuario;

    // Creamos el "payload" (la información pública que irá dentro del token)
    // Nota: "result.id" depende de cómo se llame el ID en tu entidad de usuario
    const payload = { sub: result.id, email: result.email };

    // 4. Retornamos el usuario y el token REAL firmado
    return {
      message: 'Login exitoso',
      user: result,
      // Usamos el servicio para firmar el token con tu clave 'admin123'
      access_token: this.jwtService.sign(payload),
    };
  }
}
```

Figura 2. Código fuente de auth.service.ts — implementación del método login con JWT en VS Code.

A continuación se transcribe el fragmento principal del algoritmo implementado:

```
// Fragmento principal del algoritmo de Login de Administrador
async login(loginDto: LoginDto) {
  // 1. Buscamos el usuario por su correo institucional
  const usuario = this.usuariosService.findByEmail(loginDto.email);

  // 2. Validamos usuario y contraseña
  if (!usuario || usuario.password !== loginDto.password) {
    throw new UnauthorizedException('Credenciales inválidas');
  }

  // 3. Extracción segura (sin devolver la contraseña)
  const { password, ...result } = usuario;

  // 4. Creamos el payload del token
  const payload = { sub: result.id, email: result.email };

  // 5. Retornamos usuario y token firmado
  return {
```



```
message: 'Login exitoso',
user: result,
access_token: this.jwtService.sign(payload),
};
}
```

6. Mapeo de Endpoints y Rutas del Servidor (Logs del Sistema)

Al compilar e inicializar el servidor mediante el comando `npm run start:dev`, el motor interno de NestJS (`RoutesResolver` y `RouterExplorer`) mapeó y expuso de manera exitosa los siguientes servicios en el puerto local (`http://localhost:3000`):

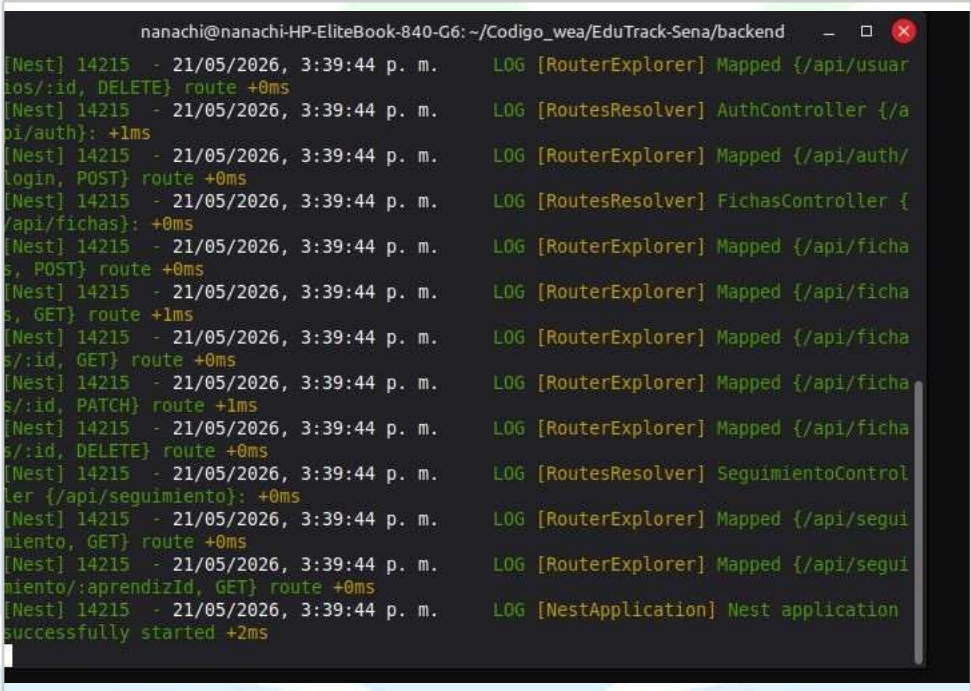


Figura 3. Logs del sistema en la terminal de Linux — NestJS mapea todos los endpoints correctamente y confirma el inicio exitoso de la aplicación.

La tabla siguiente consolida la totalidad de las rutas registradas por el sistema, junto con su método



HTTP, endpoint y propósito técnico:

Controlador de Ruta	Método HTTP	Endpoint del API	Propósito / Acción Técnica
AuthController	POST	/api/auth/login	Recibe credenciales, autentica y firma el JWT.
FichasController	GET	/api/fichas	Retorna el listado completo de fichas (200 OK).
FichasController	POST	/api/fichas	Permite la inserción de una nueva ficha académica.
FichasController	GET	/api/fichas/:id	Recupera una ficha única filtrada por su ID.

Controlador de Ruta	Método HTTP	Endpoint del API	Propósito / Acción Técnica
FichasController	PATCH	/api/fichas/:id	Modificación o actualización parcial de datos.
FichasController	DELETE	/api/fichas/:id	Eliminación lógica de un registro en la app.
SeguimientoControlle r	GET	/api/seguimiento	Consulta de bitácoras y estados de aprendices.
SeguimientoControlle r	GET	/api/seguimiento/:apre ndizId	Consulta de seguimiento por aprendiz.

■ **Nota de Inicialización:** ¡El servidor web se levantó de manera exitosa apuntando al puerto local de escucha, arrojando el mensaje base "Hello World!" al consumir la raíz del servidor en el navegador.



Figura 4. Verificación en el navegador — el servidor responde correctamente en localhost:3000/api con el mensaje base de confirmación de arranque.



7. Evidencias de Pruebas Técnicas en Postman

Caso de Prueba 01 — Consulta de Fichas Educativas (GET /api/fichas)

Se realizó una solicitud mediante el método **GET** a la dirección `http://localhost:3000/api/fichas`. El servidor respondió de manera inmediata con un estado **200 OK** en un tiempo de ejecución de **37 ms**. El cuerpo de la respuesta es un arreglo puramente estructurado en formato JSON que detalla los códigos y nombres de los programas académicos sin errores de compilación.

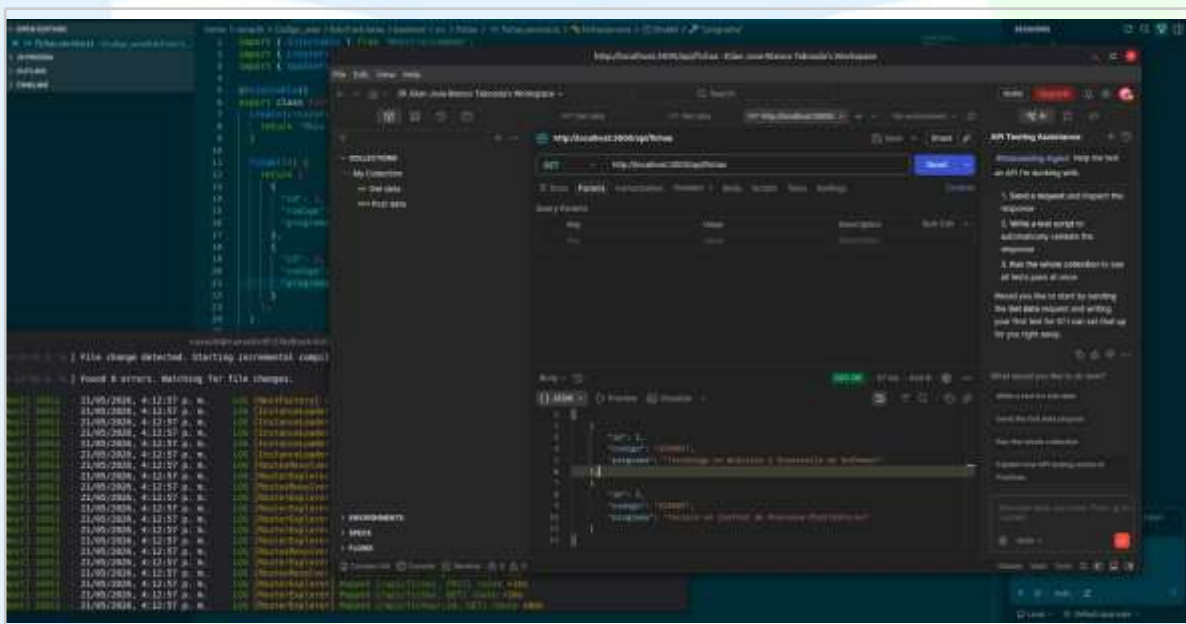


Figura 5. Postman — GET /api/fichas responde con 200 OK y el arreglo JSON de fichas académicas en 37 ms.

Caso de Prueba 02 — Autenticación Exitosa de Administrador (POST /api/auth/login)

Se realizó el envío de credenciales en el *Body* de la petición mediante el método **POST** al endpoint `/api/auth/login`.

```
{
  "email": "admin@sena.edu.co",
  "password": "admin123"
}
```

El backend de NestJS procesó la petición y devolvió un estado exitoso **201 Created** en **18 ms**. La respuesta JSON incluye el mensaje de confirmación, los datos del usuario administrador (rol: "admin", estado: true) y el token cifrado `access_token` listo para utilizarse en las cabeceras de autorización *Bearer* de los módulos subsiguientes.

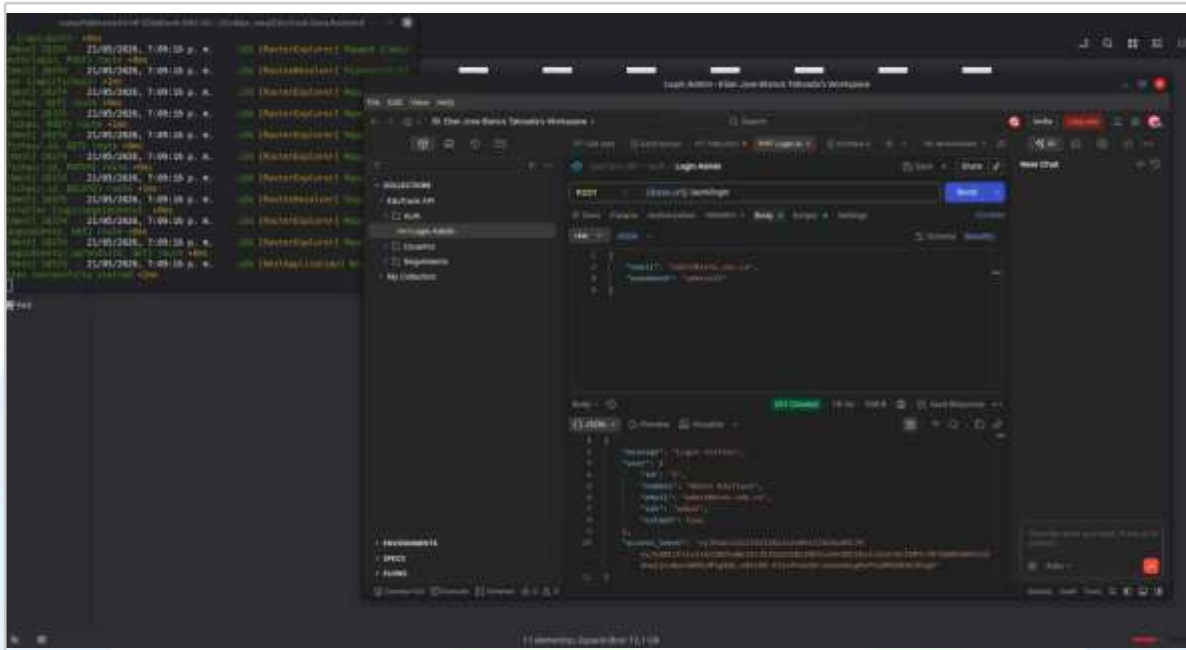


Figura 6. Vista combinada: terminal de Linux con el mapeo completo de rutas NestJS y Postman mostrando POST /api/auth/login — 201 Created con el access_token JWT generado (18 ms).

Caso de Prueba 03 — Acceso Protegido con Bearer Token (GET /api/fichas + JWT)

Una vez obtenido el `access_token` en el Caso de Prueba 02, se validó la protección de rutas mediante autenticación *Bearer Token*. Se configuró la pestaña **Authorization** de Postman con el tipo **Bearer Token** y se insertó el JWT firmado. La petición **GET** al endpoint `http://localhost:3000/api/fichas` retornó nuevamente **200 OK** en tan solo **7 ms**, confirmando que la capa de seguridad JWT funciona correctamente como guardia de acceso a los recursos protegidos de la API.

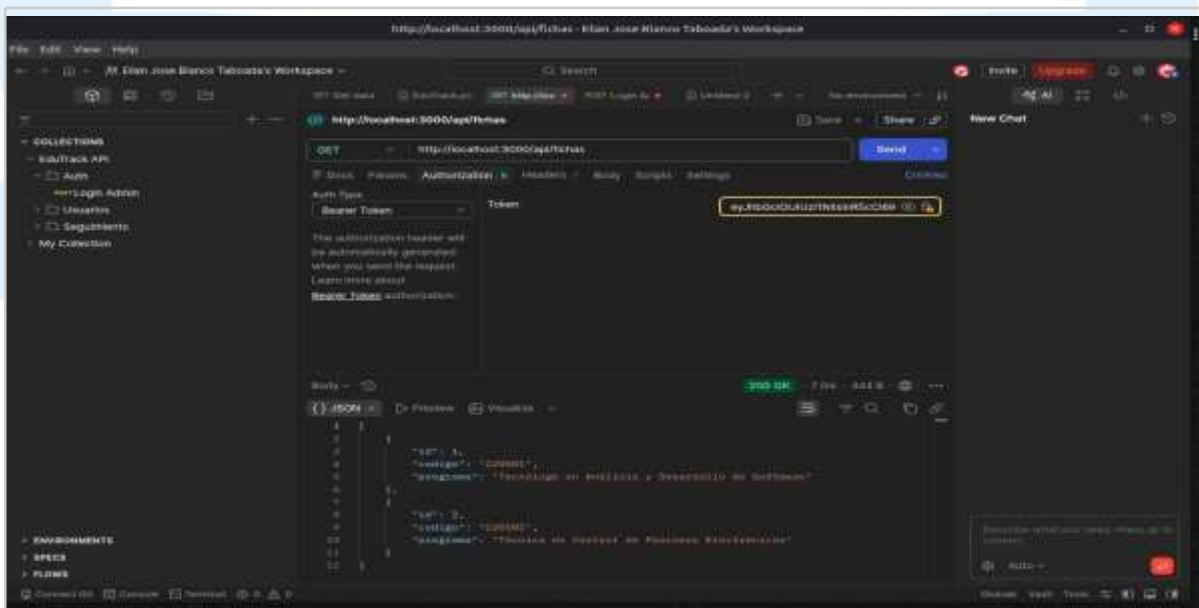


Figura 7. Postman — GET /api/fichas con autorización Bearer Token (JWT): responde 200 OK en 7 ms, demostrando el acceso autenticado y el correcto funcionamiento del guardia JWT.



8. Conclusiones

1

Operatividad de la API

La API RESTful para el proyecto **EduTrack** se encuentra completamente operativa en su capa base, respondiendo ágilmente a los verbos HTTP elementales (GET, POST, PATCH, DELETE), con tiempos de respuesta inferiores a los 40 ms en todas las pruebas realizadas.

2

Seguridad con JWT

La implementación del estándar **JSON Web Token (JWT)** asegura que el intercambio de información futura en servicios en la nube cuente con capas restrictivas y seguras de acceso, cumpliendo con las buenas prácticas de autenticación sin estado (stateless).

3

Entrega de Evidencias

El código fuente íntegro ha sido empaquetado y depurado dentro de un archivo contenedor comprimido con extensión `.zip`, cumpliendo rigurosamente con los lineamientos de entrega y formato libre exigidos por la guía de actividades **GA6-220501123-AA2-EV01**.

4

Cumplimiento de la Guía

Todos los requisitos técnicos indicados en la guía — API RESTful, intercambio de datos en formato JSON y validación a través de Postman — han sido satisfechos en su totalidad, conforme a los conceptos de buenas prácticas de desarrollo y arquitecturas escalables para la nube.



8. Referencias y Bibliografía

Para el diseño, codificación, aseguramiento y validación de la API RESTful del proyecto "EduTrack", se consultaron las siguientes fuentes oficiales y estándares de la industria de desarrollo de software:

- **NestJS Core Documentation – Authentication & Security** Consultado para la implementación de la arquitectura modular, uso de decoradores (`@Injectable()`, `@Controller()`), inyección de dependencias (`AuthService`, `UsuariosService`) y la integración del módulo `@nestjs/jwt` mediante *guards* y estrategias de control de acceso.

Enlace oficial: <https://docs.nestjs.com/security/authentication>

- **JSON Web Token (JWT) - RFC 7519 Standard** Utilizado como fundamento teórico para la estructuración segura del *payload* (`sub`, `email`), el firmado digital sin estado (*stateless authentication*) y el manejo de tokens de acceso eficientes para arquitecturas escalables en la nube.

Enlace oficial: <https://datatracker.ietf.org/doc/html/rfc7519>

- **REST API Design Best Practices & HTTP Status Codes** Fuente de referencia para la correcta asignación de métodos HTTP (GET, POST, PATCH, DELETE) y el mapeo de códigos de estado del servidor en la plataforma Postman (como el retorno de 200 OK para consultas exitosas y 201 Created para la generación de tokens de autenticación).

Enlace oficial (MDN Web Docs): <https://developer.mozilla.org/es/docs/Web/HTTP/Status>

- **TypeScript Language Specification & NestJS CLI** Documentación base empleada para la configuración del entorno de desarrollo global en la estación de trabajo Linux Ubuntu, el tipado estático de objetos JSON, y el control de logs mediante comandos de compilación en tiempo real (`npm run start:dev`).

Enlace oficial: <https://www.typescriptlang.org/docs/>